Constrained Delaunay tetrahedrisation refinement

Royer Max

CNR IMATI

July 22, 2025

Outline

Introduction
- Who am I?
- Constrained Delaunay tetrahedrisation
- State of the art

The refinement algorithm
- General principle
- Edge collapse
- A new operation

Results
- Some graphics
- Comparison to TetGen

Conclusion

Quick presentation

- Max Royer

Figure: A ragu I made in Genova

Quick presentation

- Max Royer
- ENS of Lyon

Figure: A ragu I made in Genova

Quick presentation

- Max Royer
- ENS of Lyon
- End of first year of master

Figure: A ragu I made in Genova

Quick presentation

- Max Royer
- ENS of Lyon
- End of first year of master
- Internship for 3 months

Figure: A ragu I made in Genova

Let's add some constraints

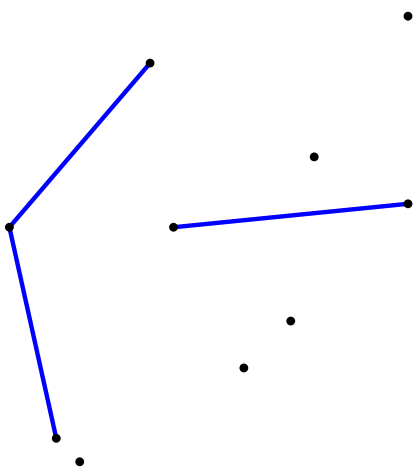


Figure: Edge constraints for Delaunay triangulation

Let's add some constraints

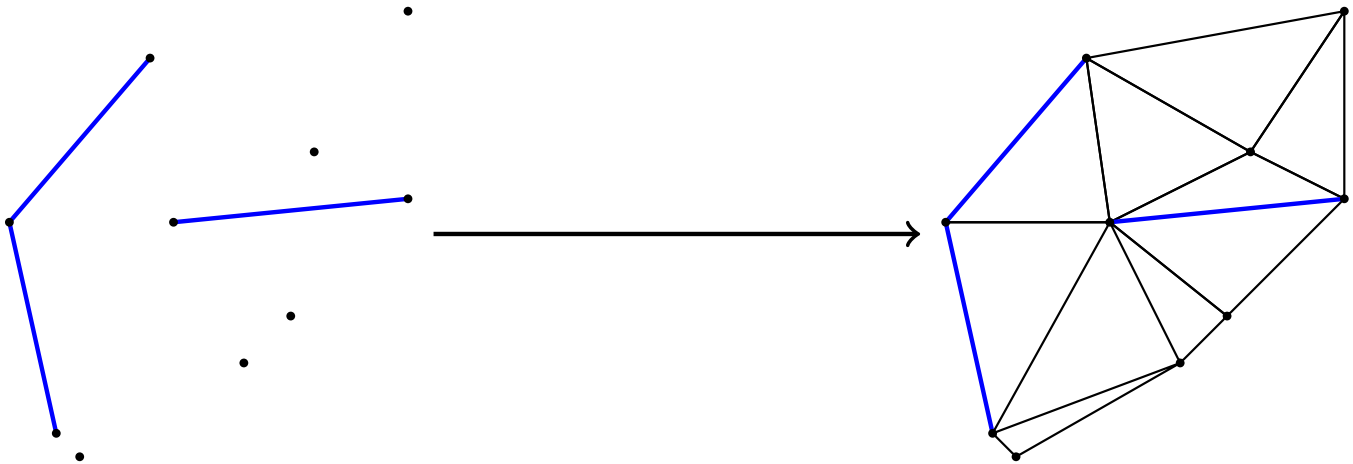


Figure: Edge constraints for Delaunay triangulation

Let's add some constraints

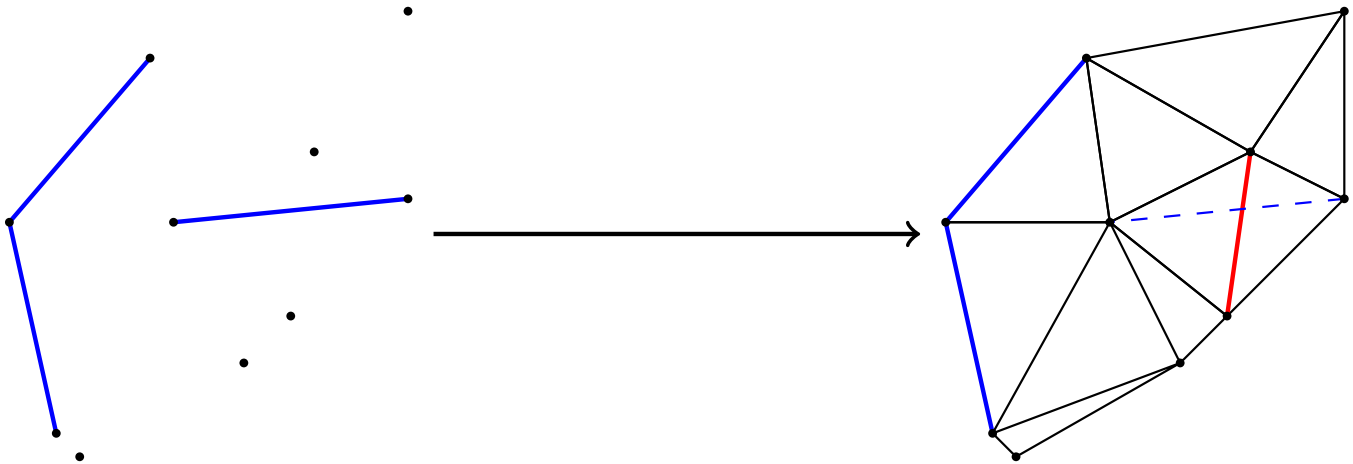


Figure: Edge constraints for Delaunay triangulation

Let's add some constraints

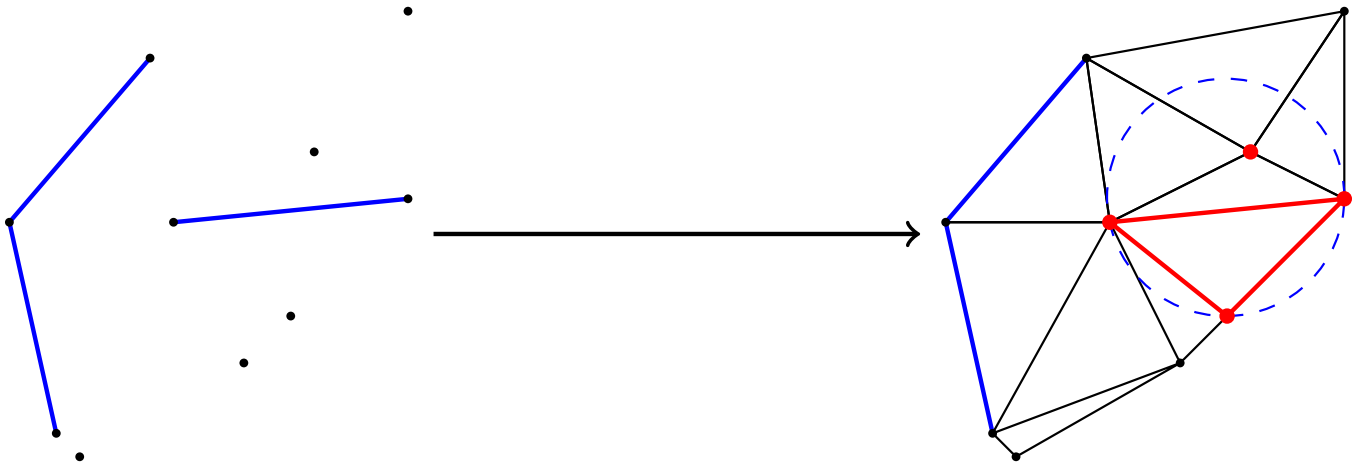


Figure: Edge constraints for Delaunay triangulation

Let's add some constraints

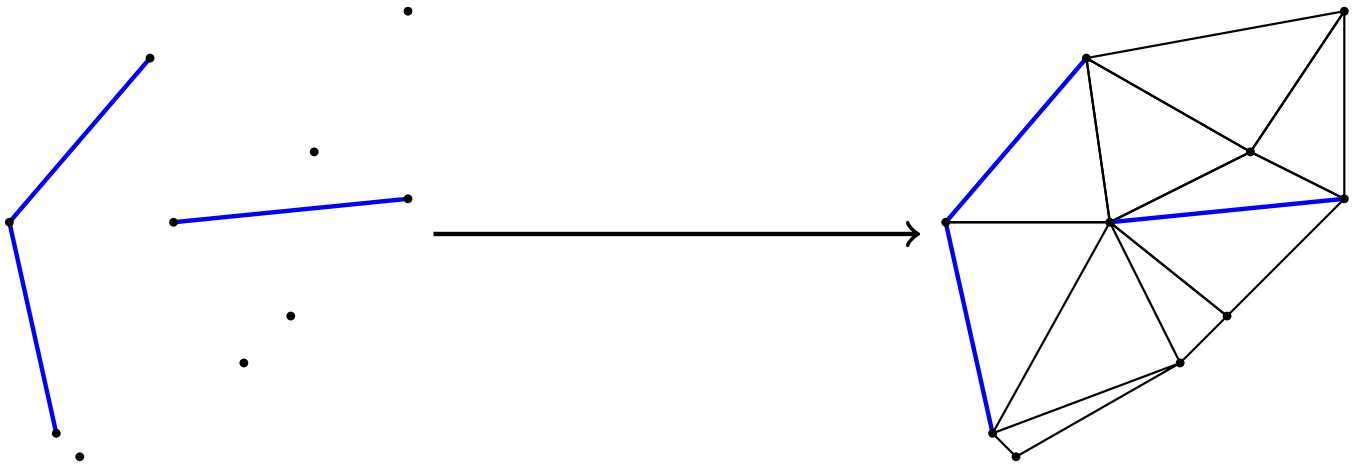
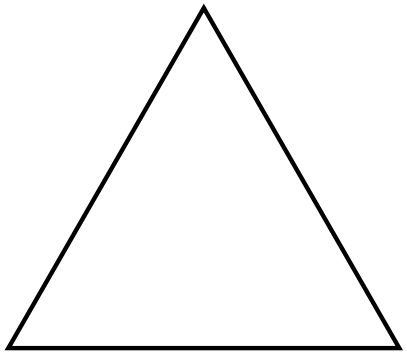


Figure: Edge constraints for Delaunay triangulation

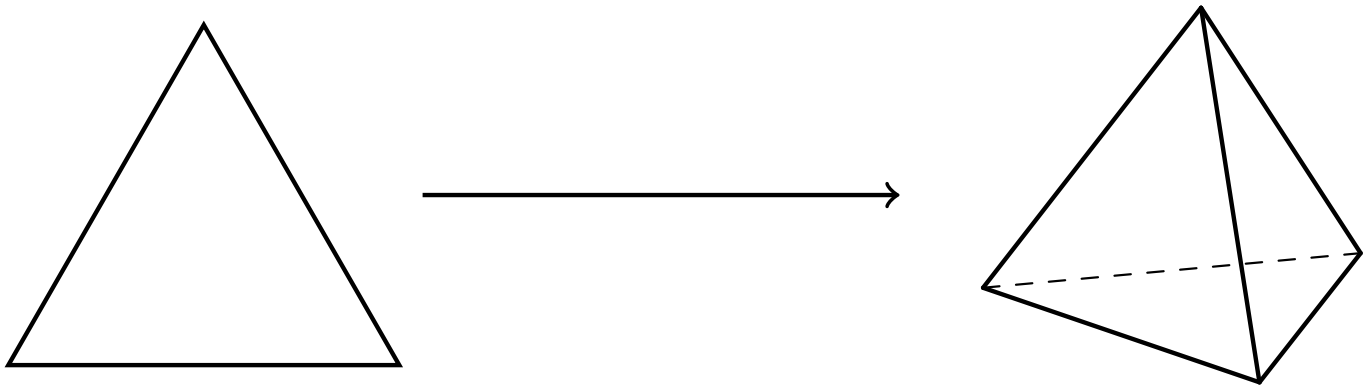
An edge is either a constraint or is locally Delaunay

Differences from 2D to 3D?



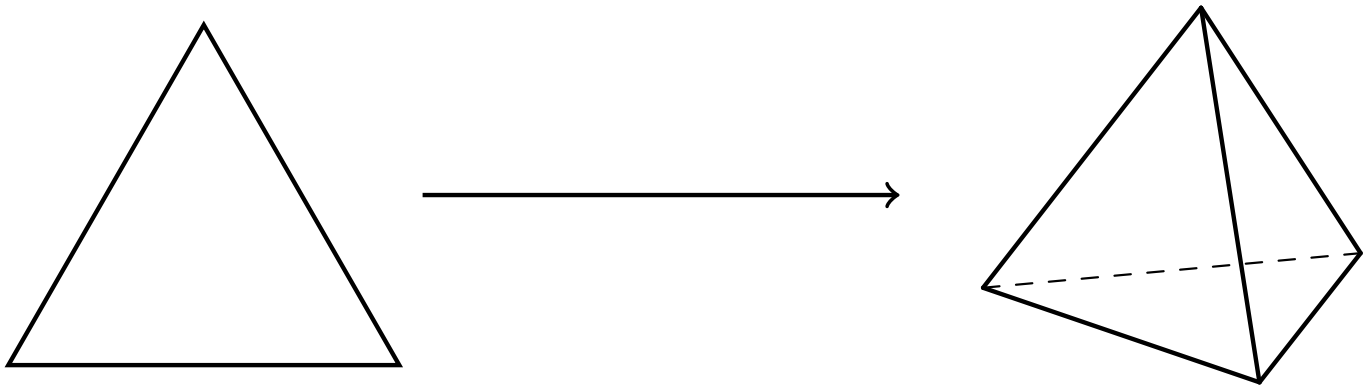
From this ...

Differences from 2D to 3D?



From this ... ... to this

Differences from 2D to 3D?



From this ... ... to this

Only ??

Not really...

... it gets harder

Not really...

... it gets harder

- Slivers appear

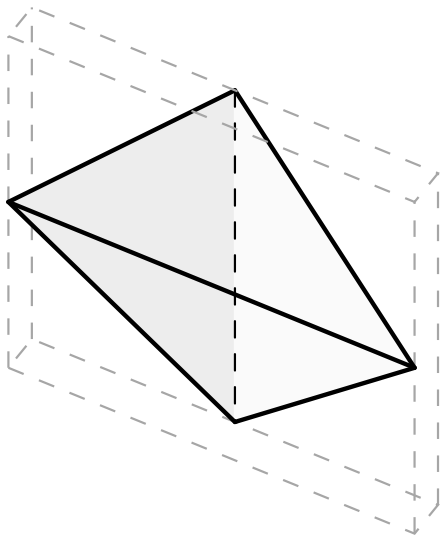


Figure: Example of sliver

Not really...

... it gets harder

- Slivers appear

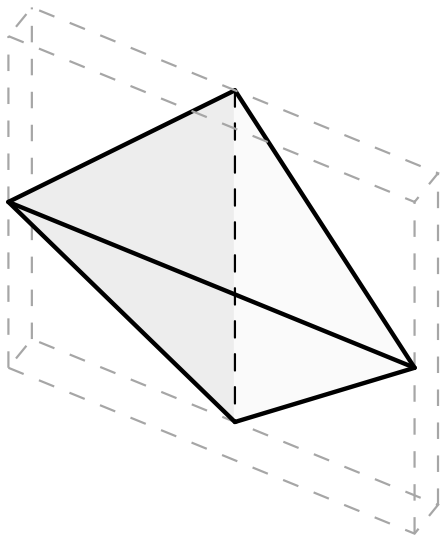


Figure: Example of sliver

- Constrained Delaunay tetrahedrisation are not always possible

TetGen [Si, 2015]

Already exisiting software

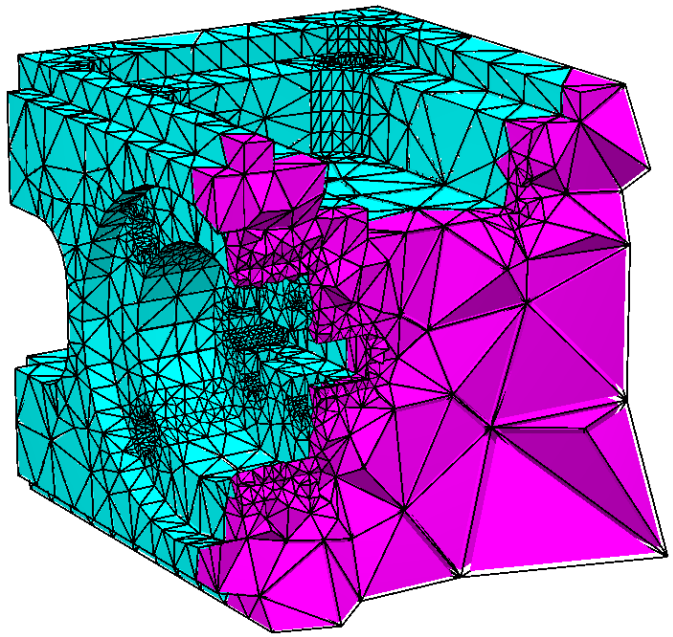


Figure: A model tetrahedrized by TetGen

- Not so robust, crashs on some models

Tetrahedral meshing in the wild (Tetwild) [Hu, 2018]

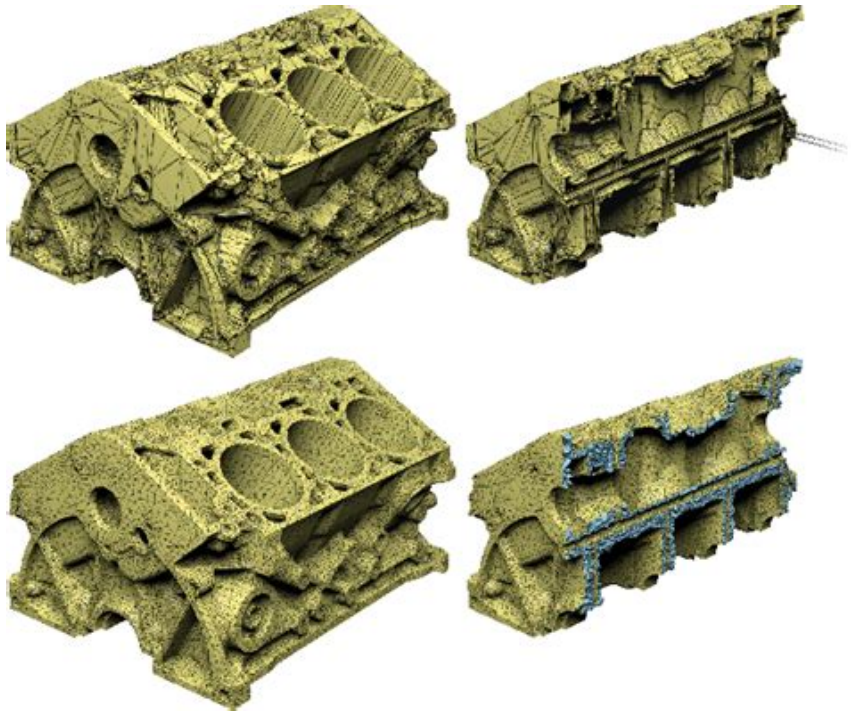


Figure: A model tetrahedrized by Tetwild

- Not strict on boundary

Quality measure

How to say if a tetrahedron is *"good"*?

Quality measure

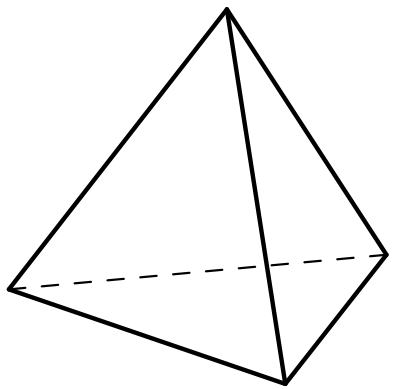
How to say if a tetrahedron is *"good"*?

- *Dirichlet* energy

Quality measure

How to say if a tetrahedron is *"good"*?

- *Dirichlet* energy

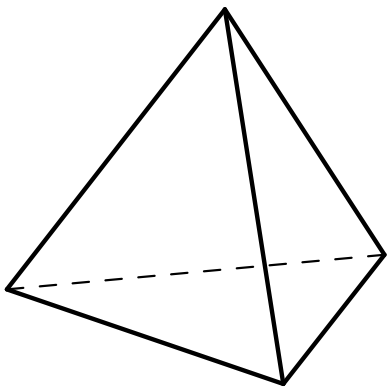


Minimized on a regular tetrahedron ($= 3$)

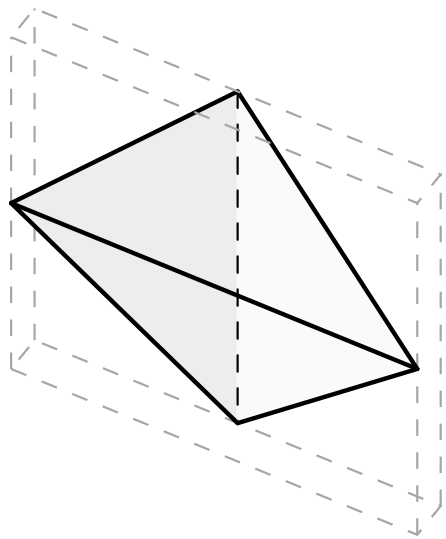
Quality measure

How to say if a tetrahedron is *"good"*?

- *Dirichlet* energy



Minimized on a regular tetrahedron ($= 3$)



Big value on bad tetrahedron ($\sim 10^3 - 10^4$)

Optimization Pass

Figure: Optimization process scheme

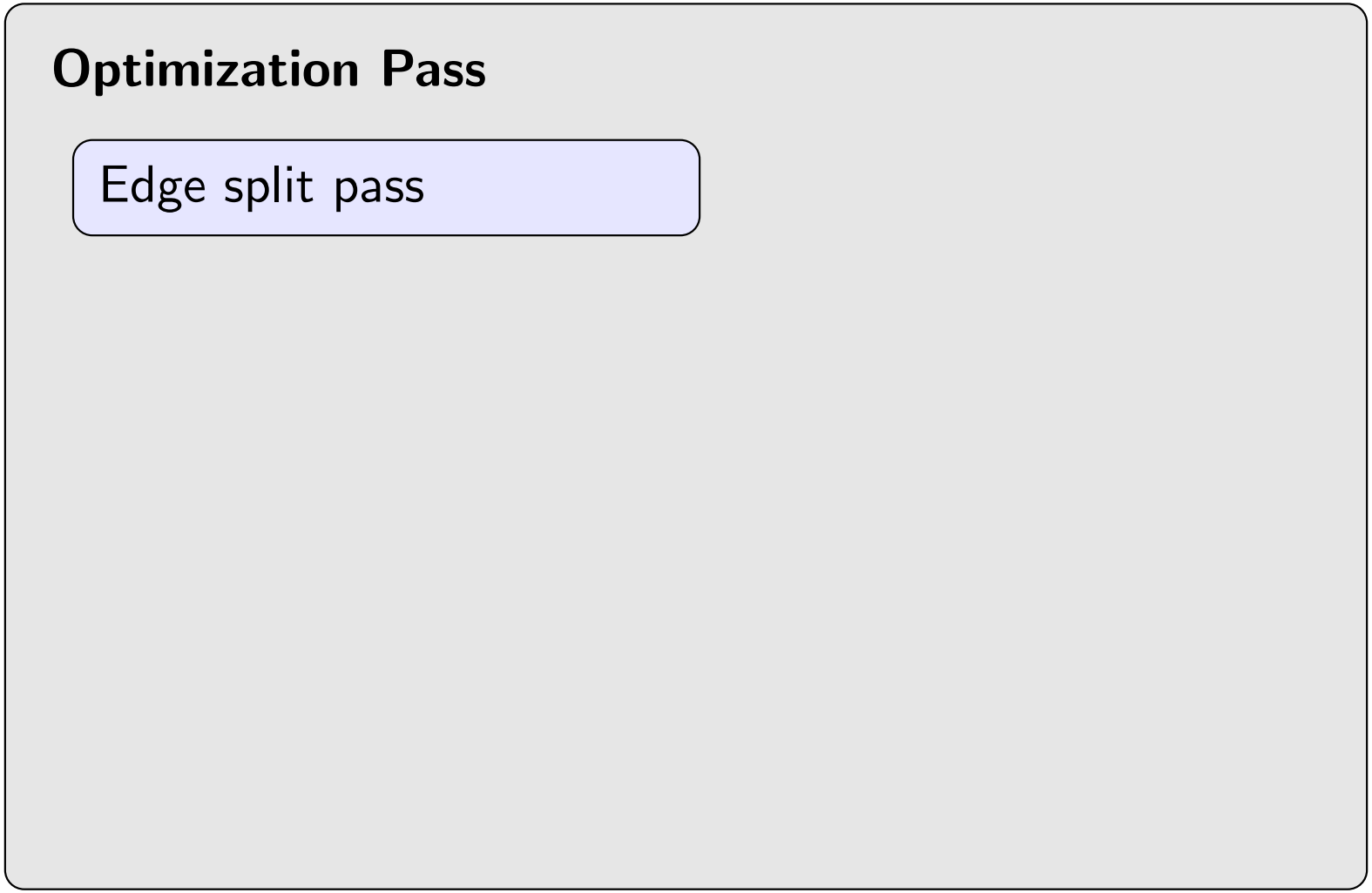


Figure: Optimization process scheme

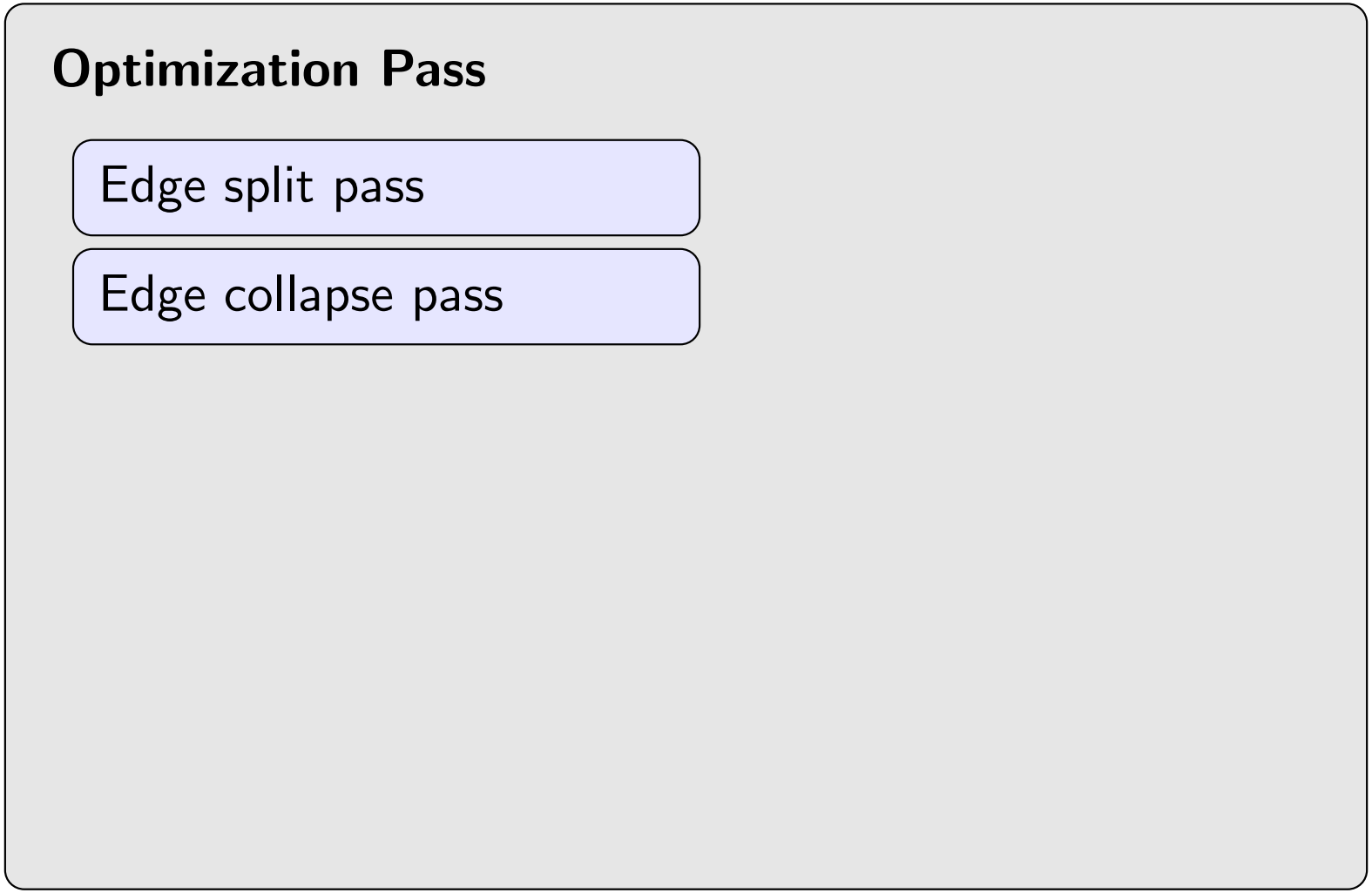


Figure: Optimization process scheme

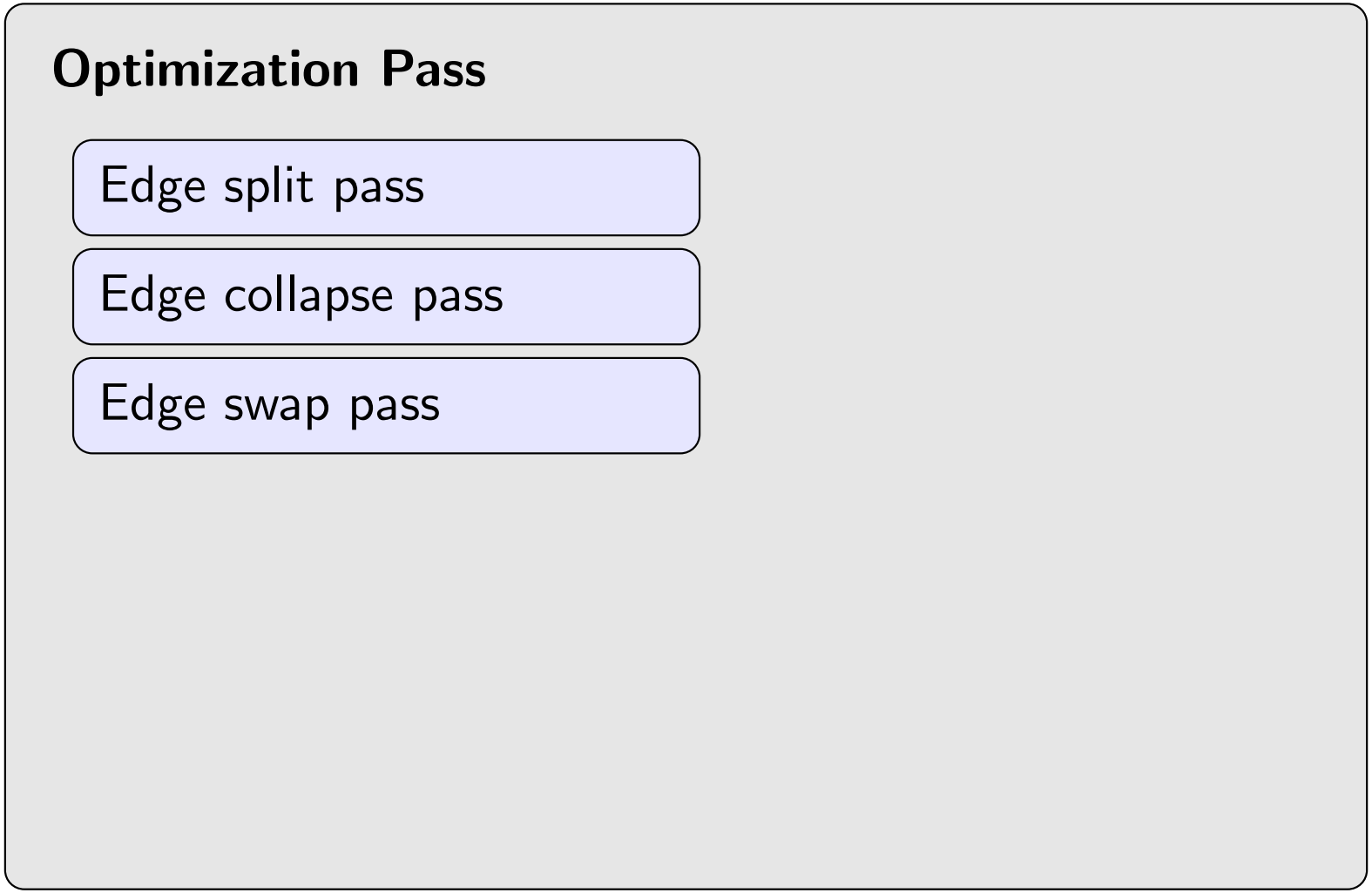


Figure: Optimization process scheme

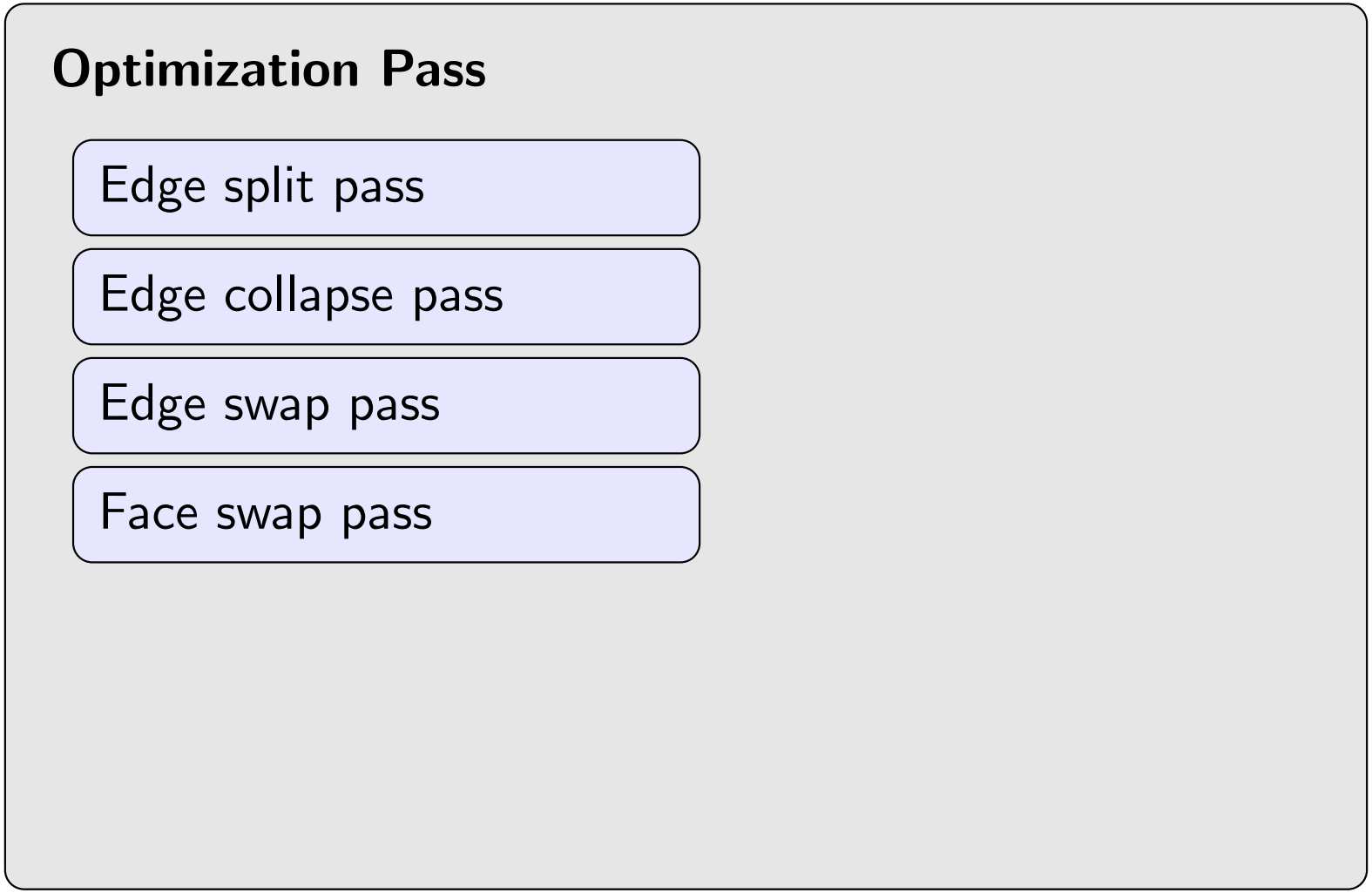


Figure: Optimization process scheme

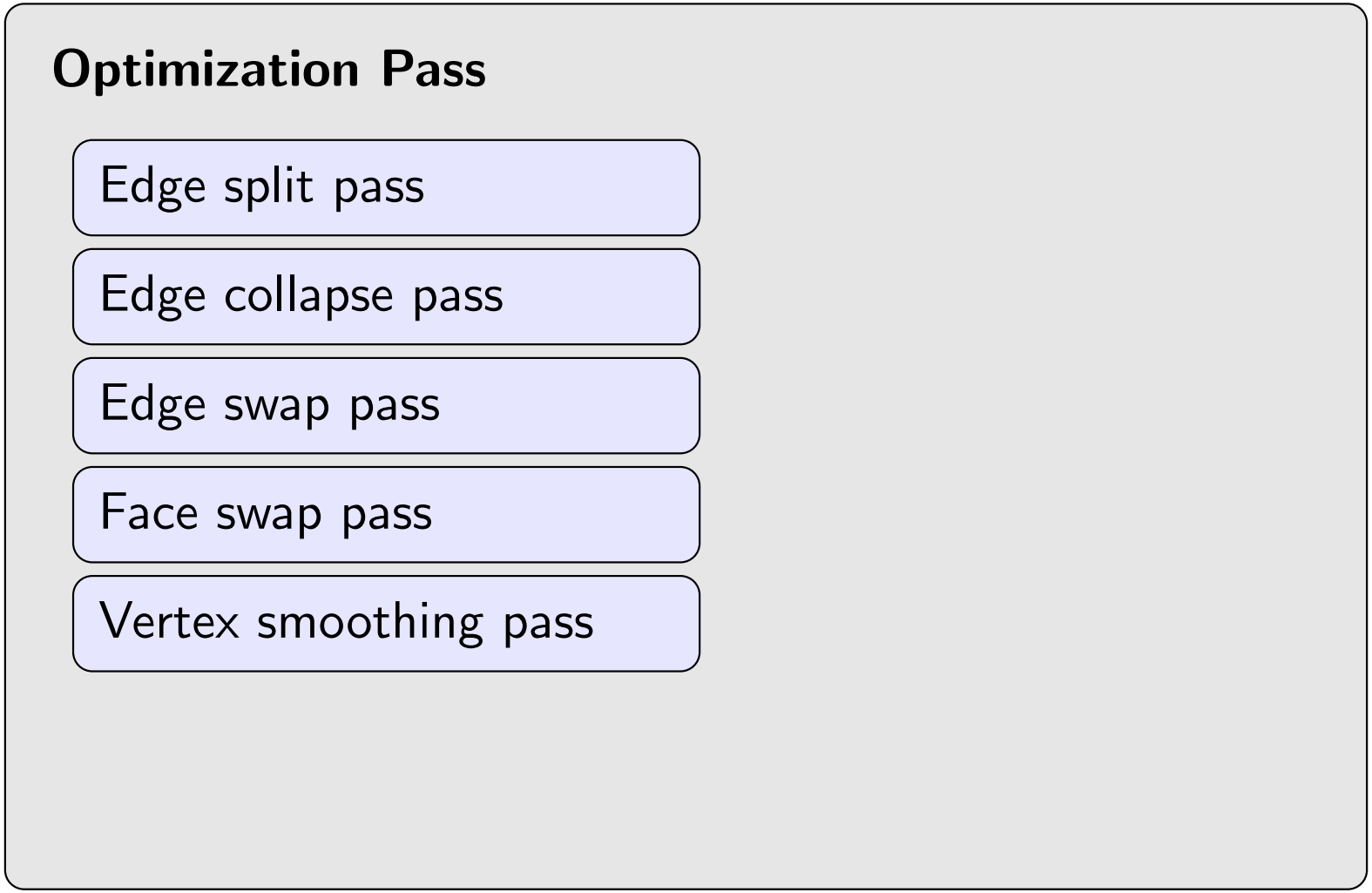


Figure: Optimization process scheme

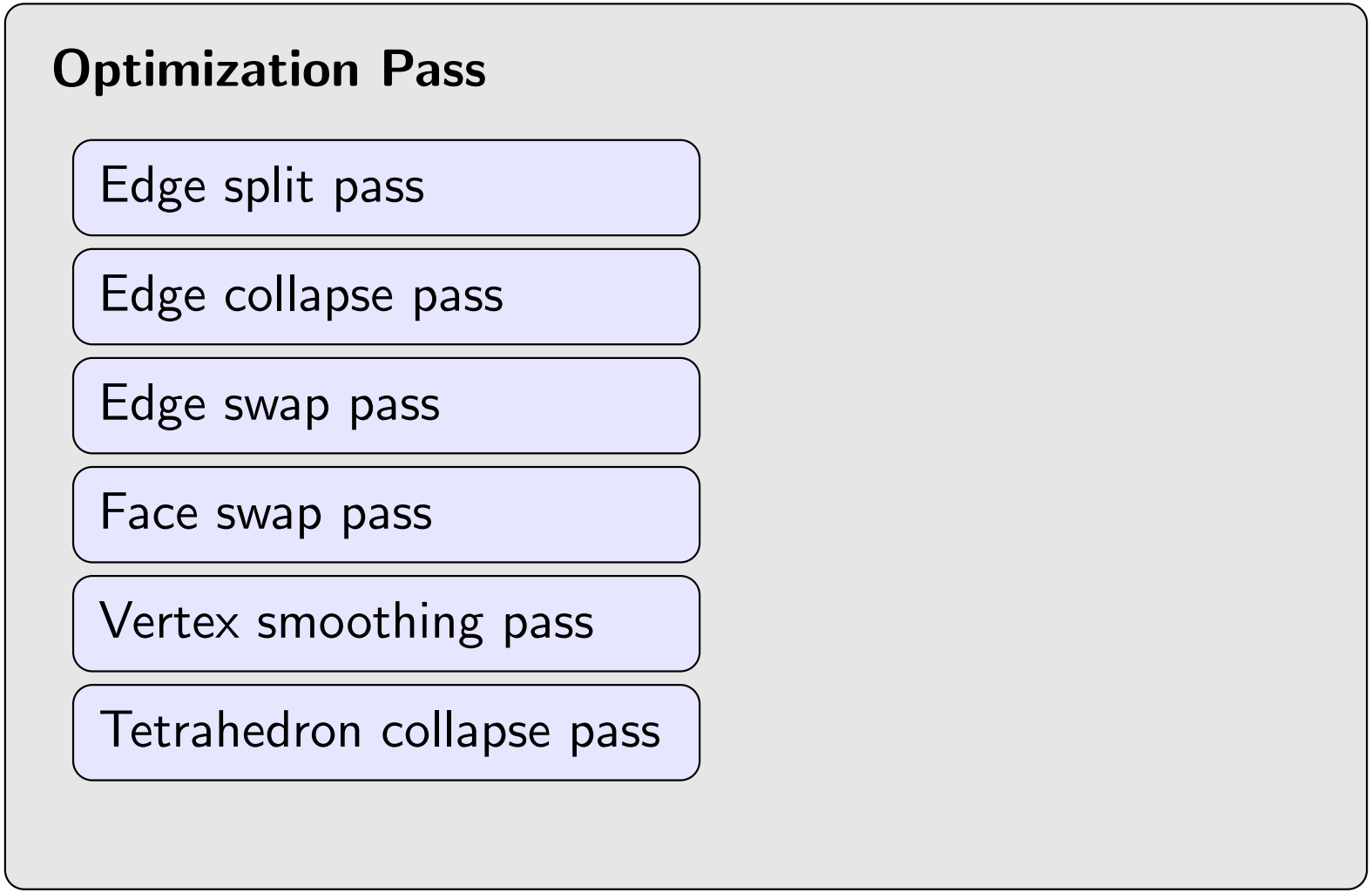


Figure: Optimization process scheme

Local operation pass

Figure: Local operation scheme

Local operation pass

Simulation pass

Figure: Local operation scheme

Local operation pass

Simulation pass $\longrightarrow$ Yields a queue Q

Figure: Local operation scheme

Local operation pass

Simulation pass $\longrightarrow$ Yields a queue Q

Applying pass, while Q not empty:

Figure: Local operation scheme

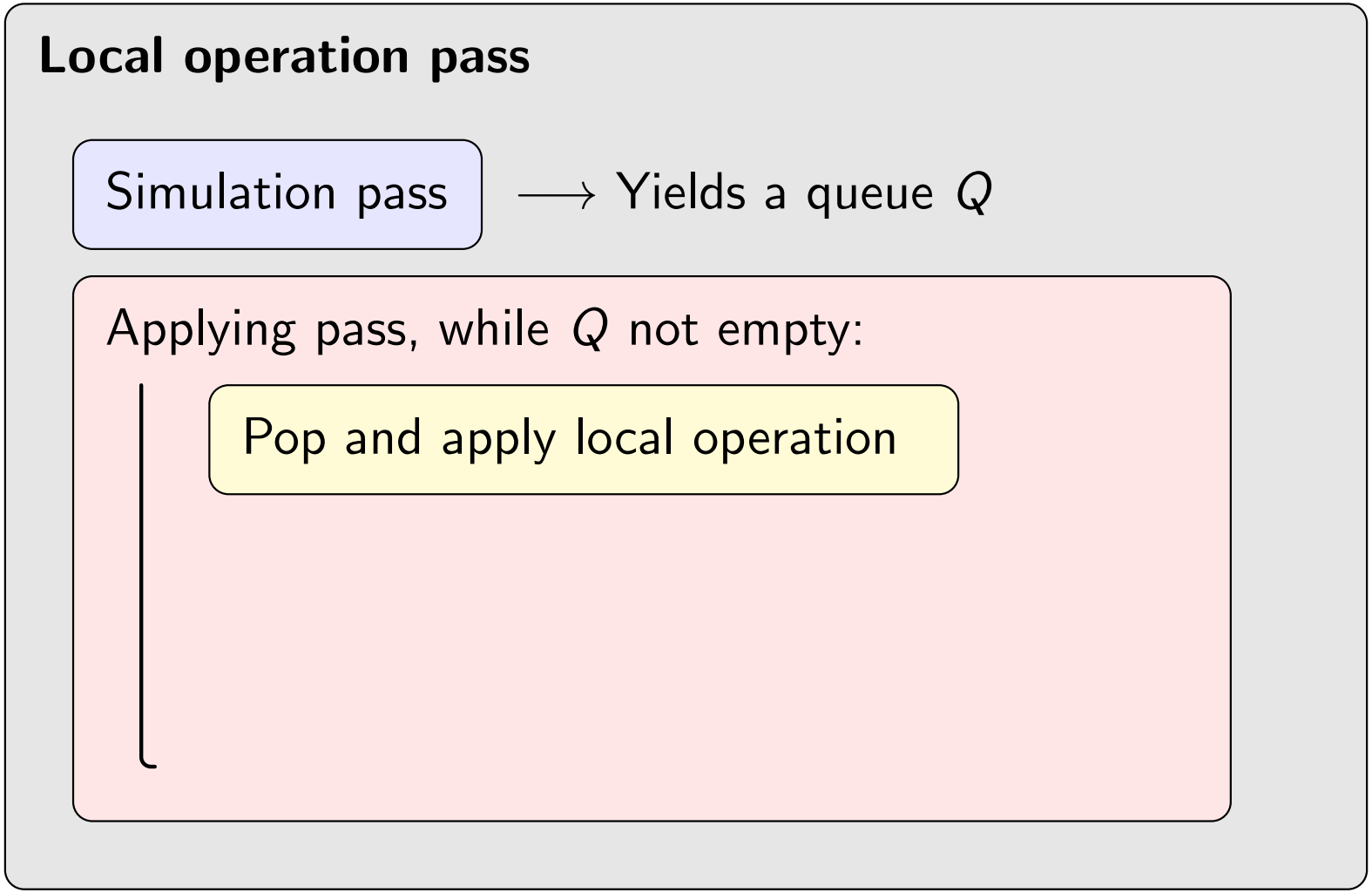


Figure: Local operation scheme

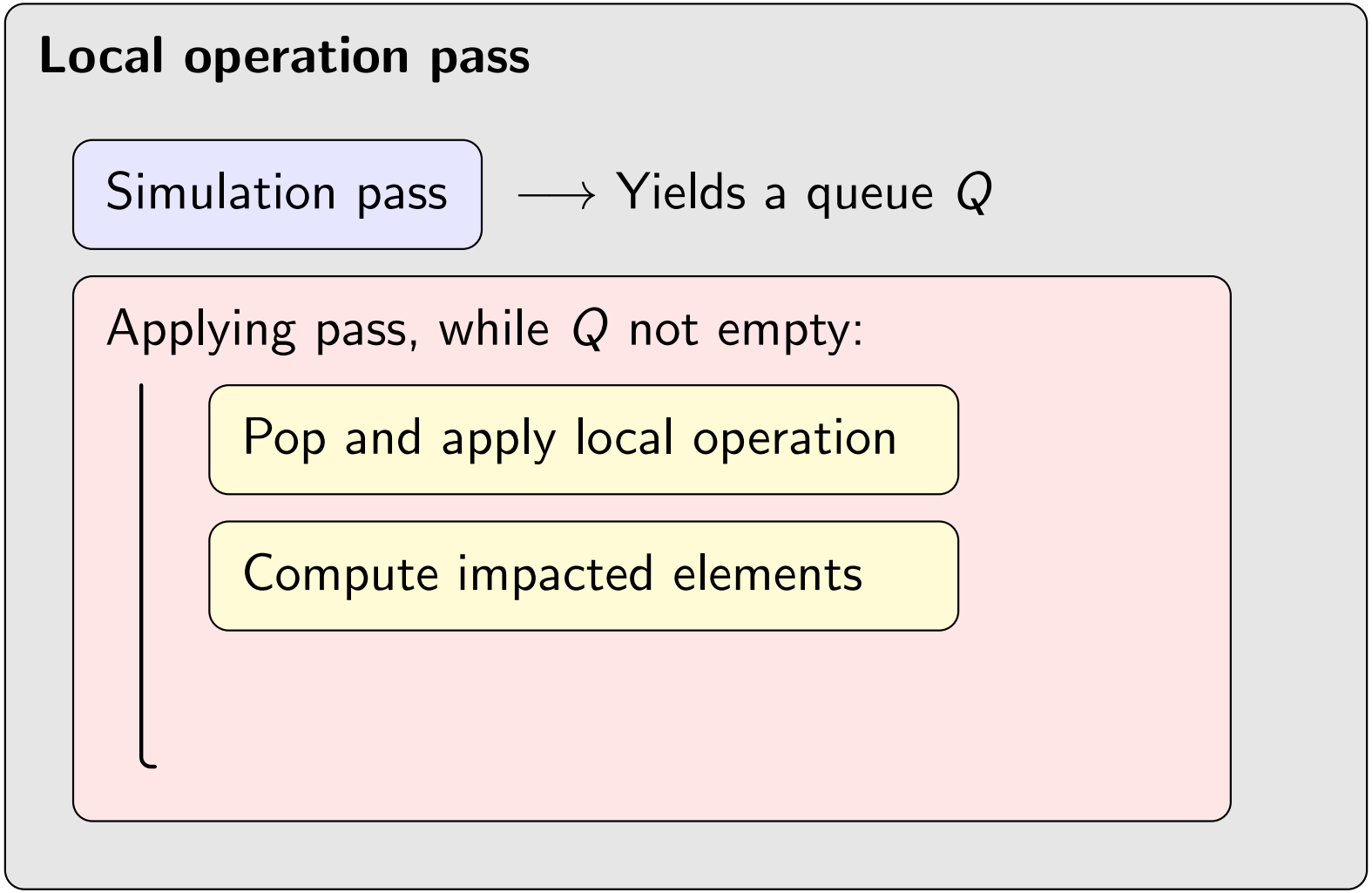


Figure: Local operation scheme

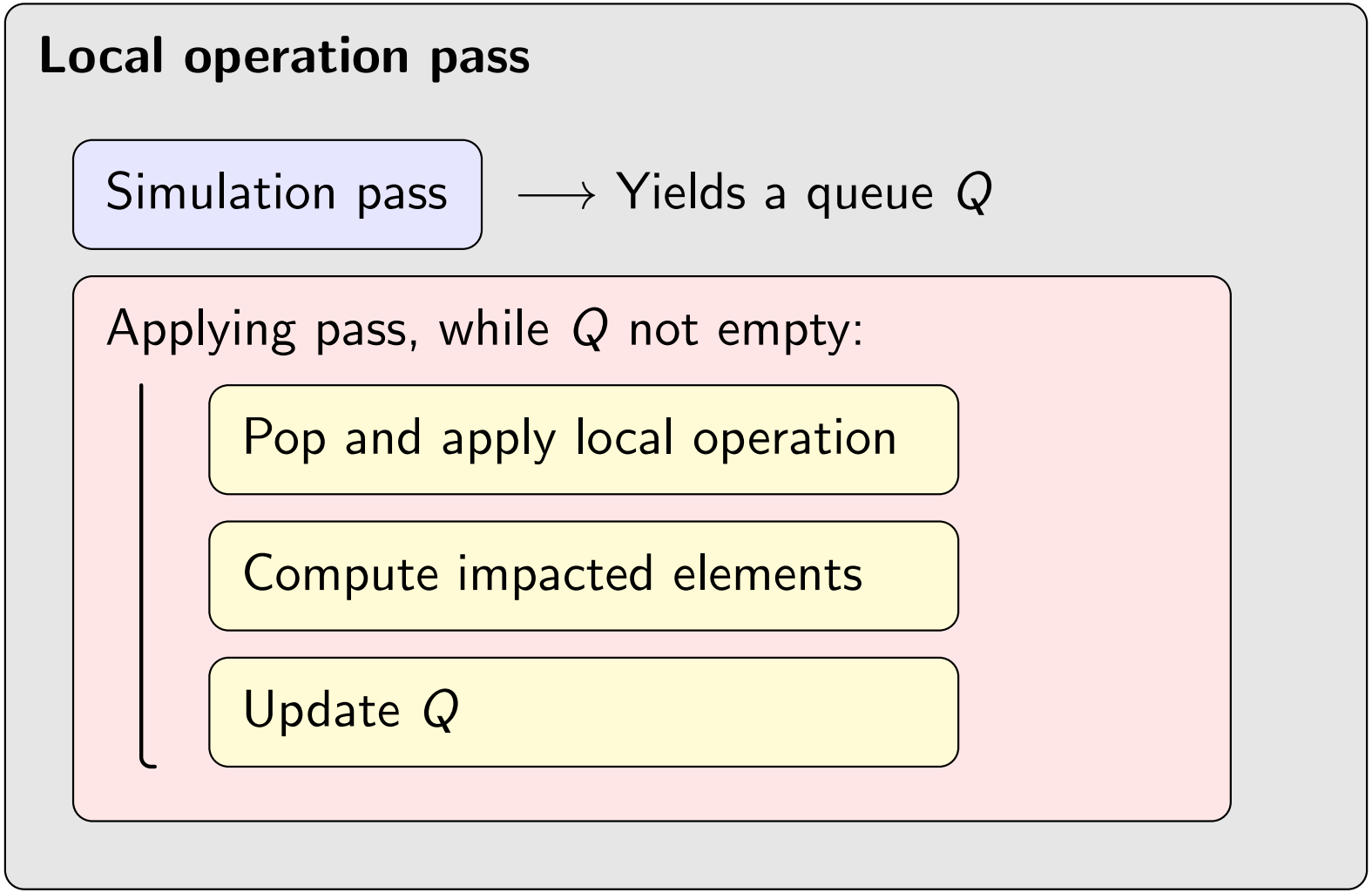


Figure: Local operation scheme

Overview

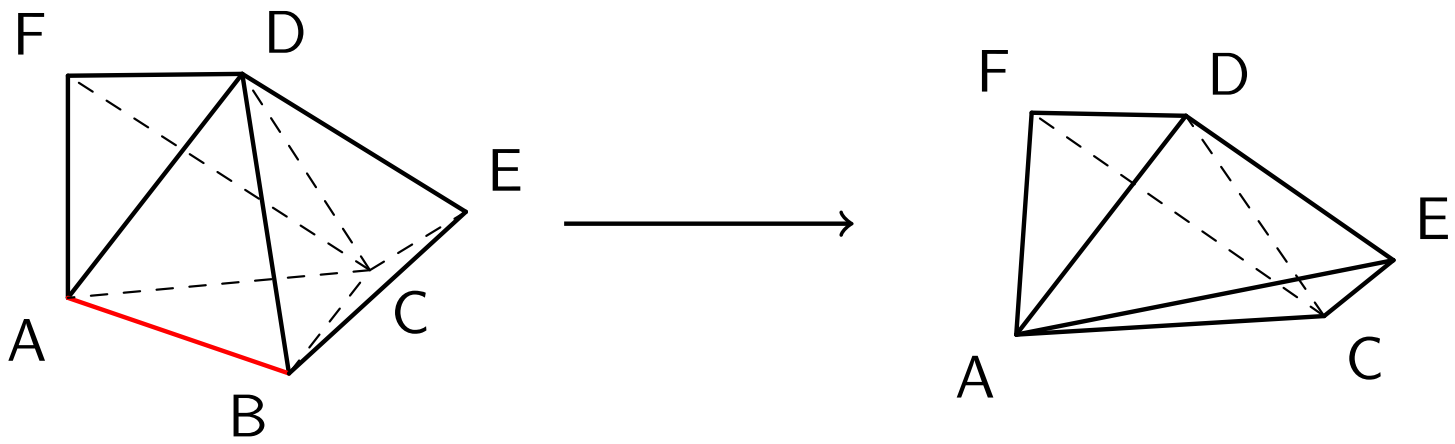


Figure: Example of collapsing an edge

Simulation

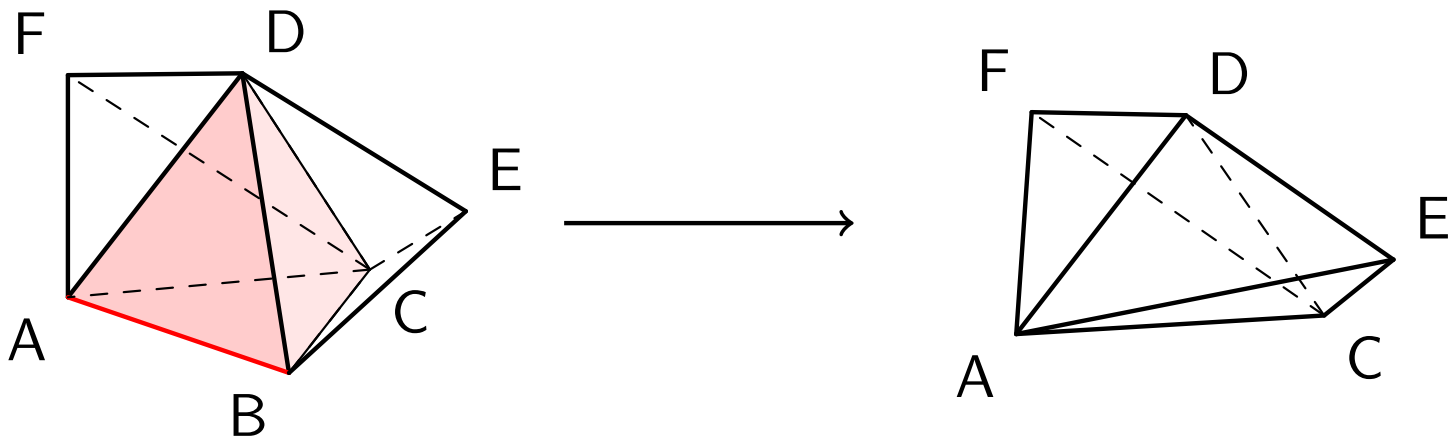


Figure: Impacted tetrahedra by an edge collapse

Simulation

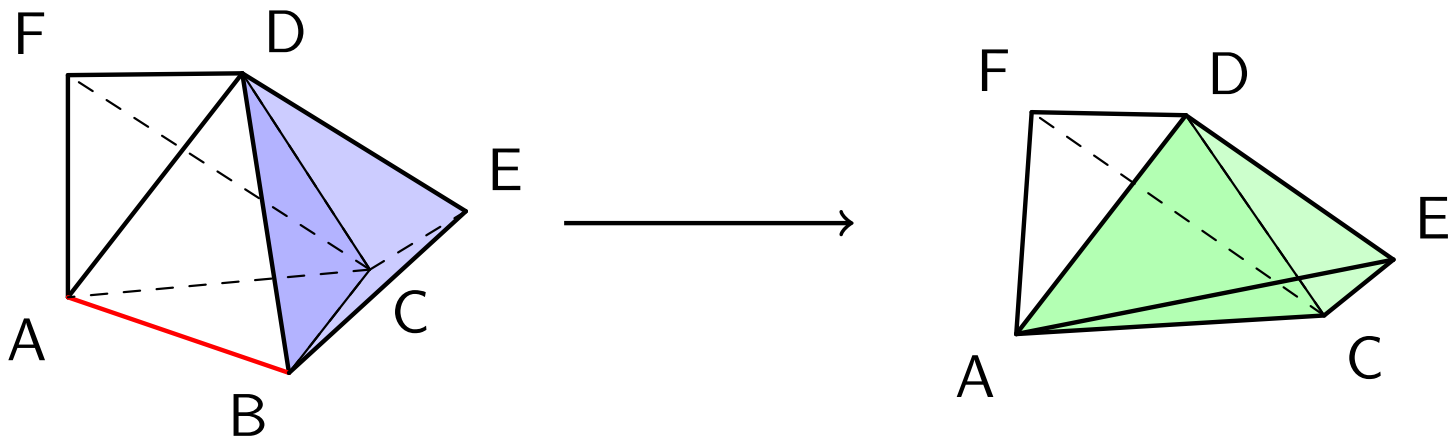


Figure: Impacted tetrahedra by an edge collapse

Simulation

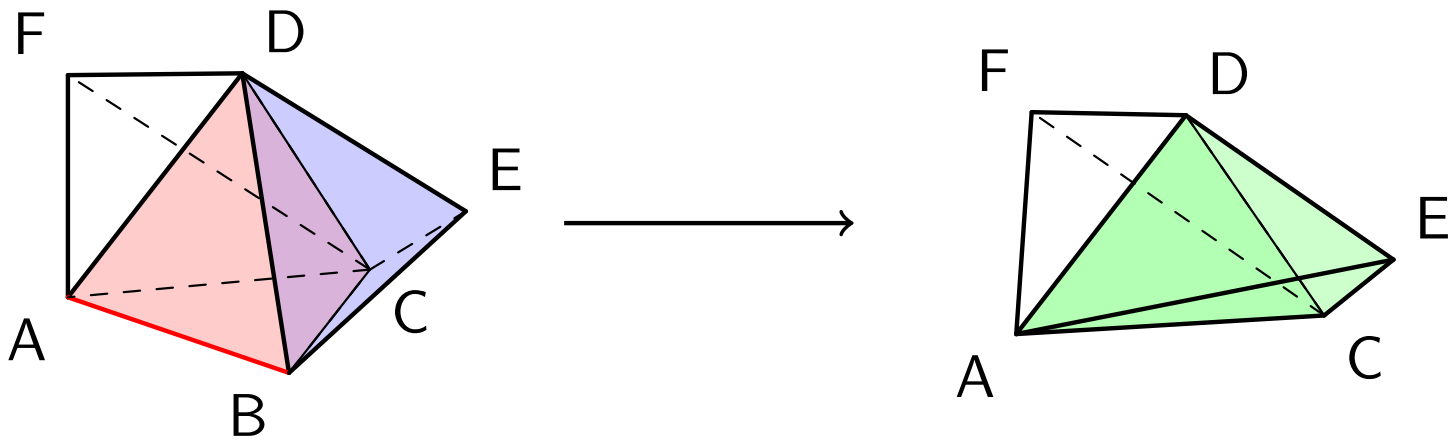


Figure: Impacted tetrahedra by an edge collapse

Quality before:

Max on removed elements ∪
unupdated elements

Simulation

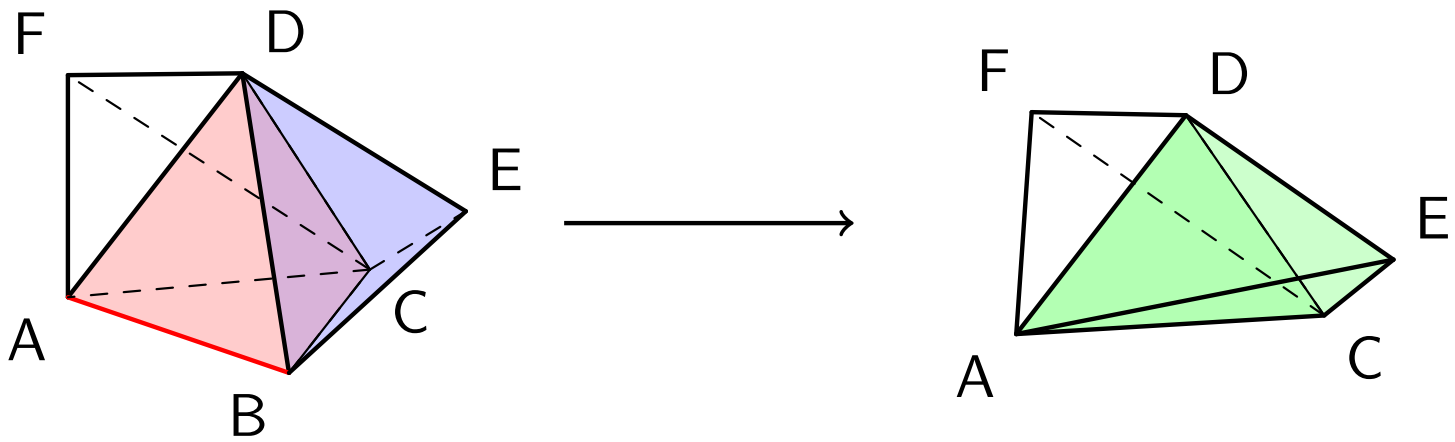


Figure: Impacted tetrahedra by an edge collapse

Quality before:

Max on removed elements ∪ unupdated elements

Quality after:

Max on updated elements

Topological failure

Collapsing $e := (u, v)$ can break topology

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

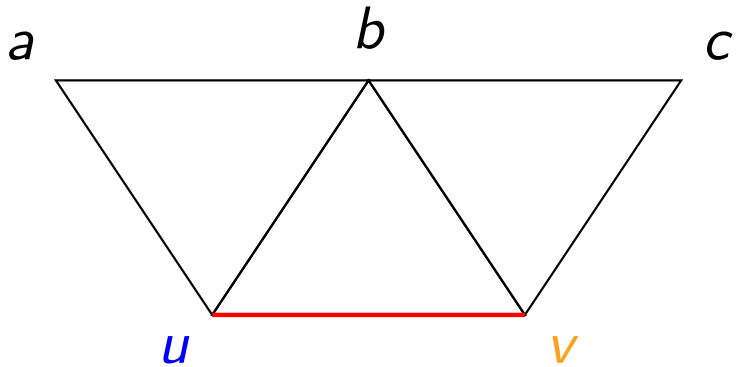


Figure: Respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

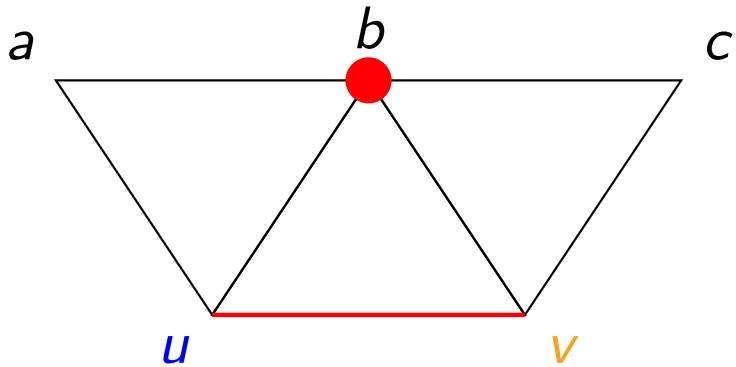


Figure: Respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

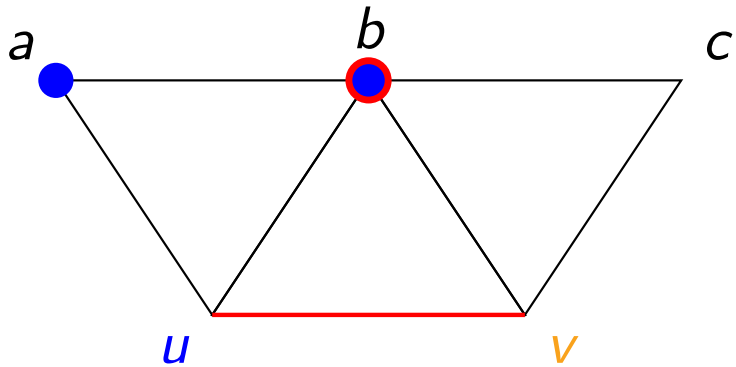


Figure: Respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

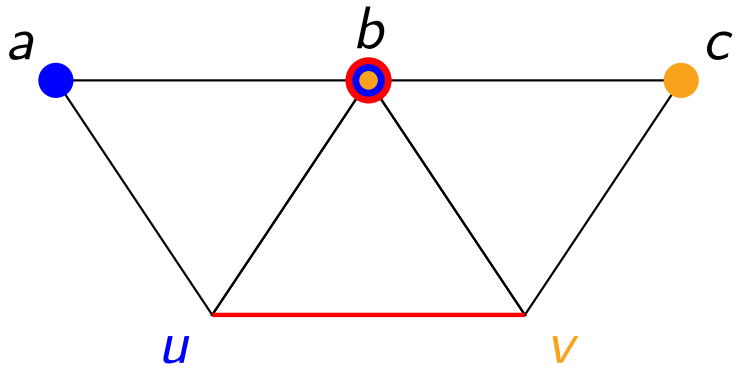


Figure: Respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

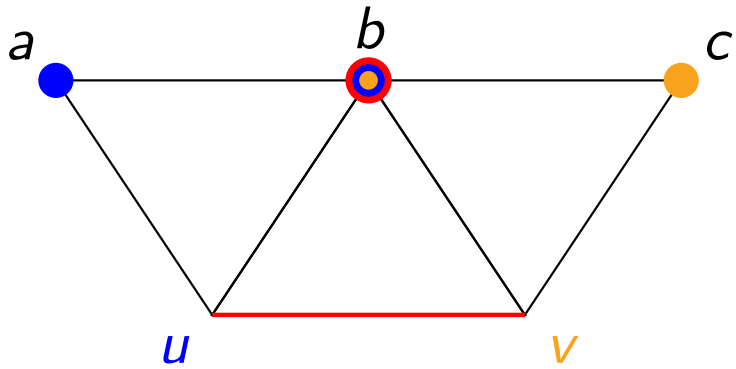


Figure: Respected link condition

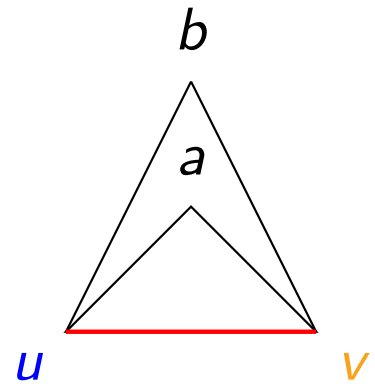


Figure: Not respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

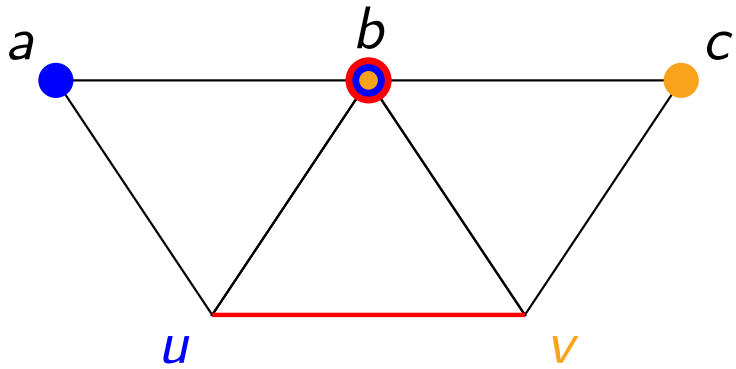


Figure: Respected link condition

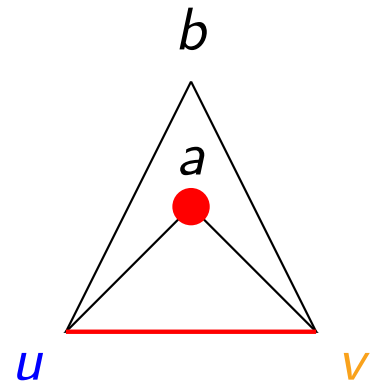


Figure: Not respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

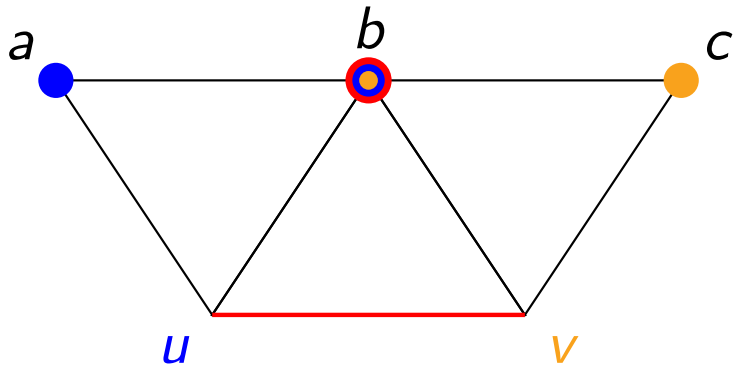


Figure: Respected link condition

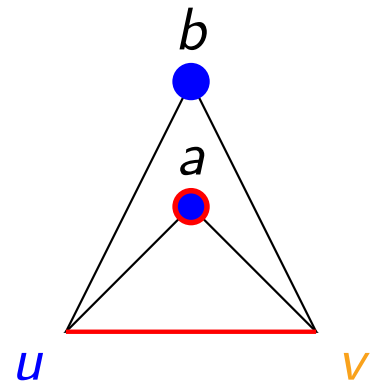


Figure: Not respected link condition

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\mathrm{Lk}(e) = \mathrm{Lk}(u) \cap \mathrm{Lk}(v) \qquad \text{(Link condition)}$$

- $\mathrm{Lk}(u) =$ neighbours of u
- $\mathrm{Lk}(e) =$ vertices sharing a face with e

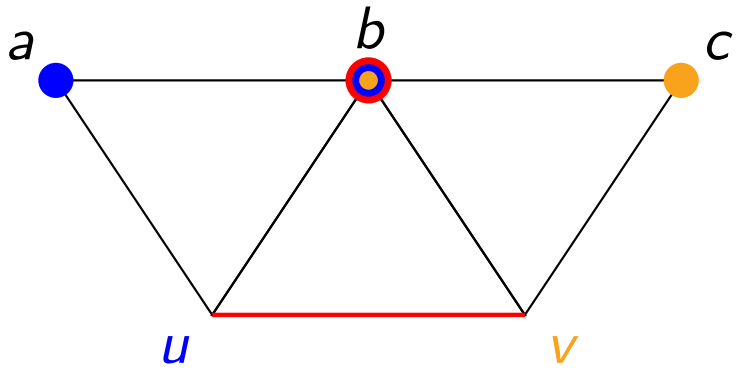


Figure: Respected link condition

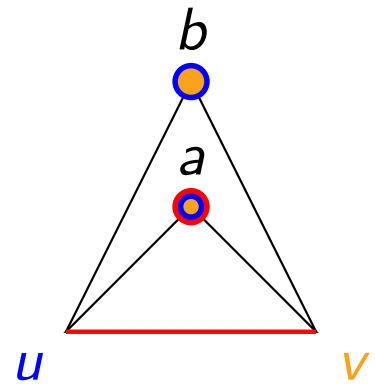


Figure: Not respected link condition

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation ⇒ Create a new operation

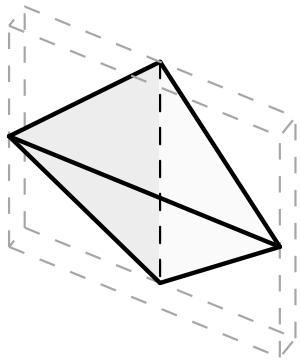


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation ⇒ Create a new operation

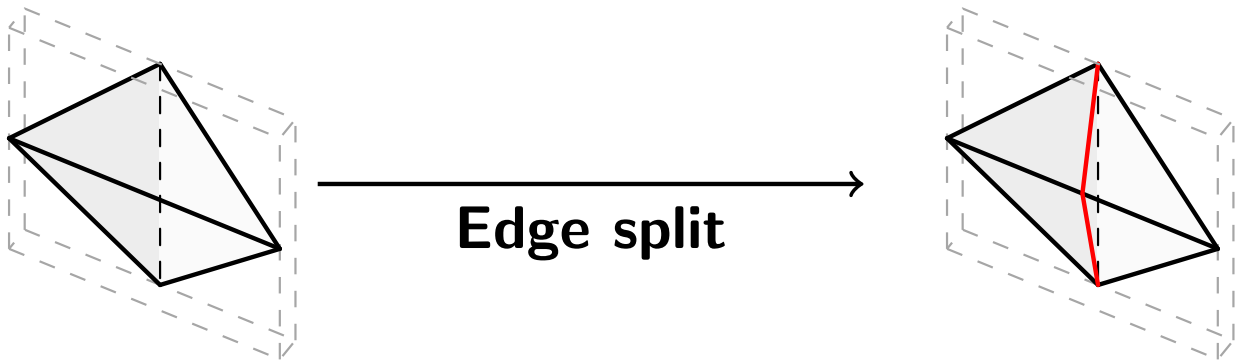


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation ⇒ Create a new operation

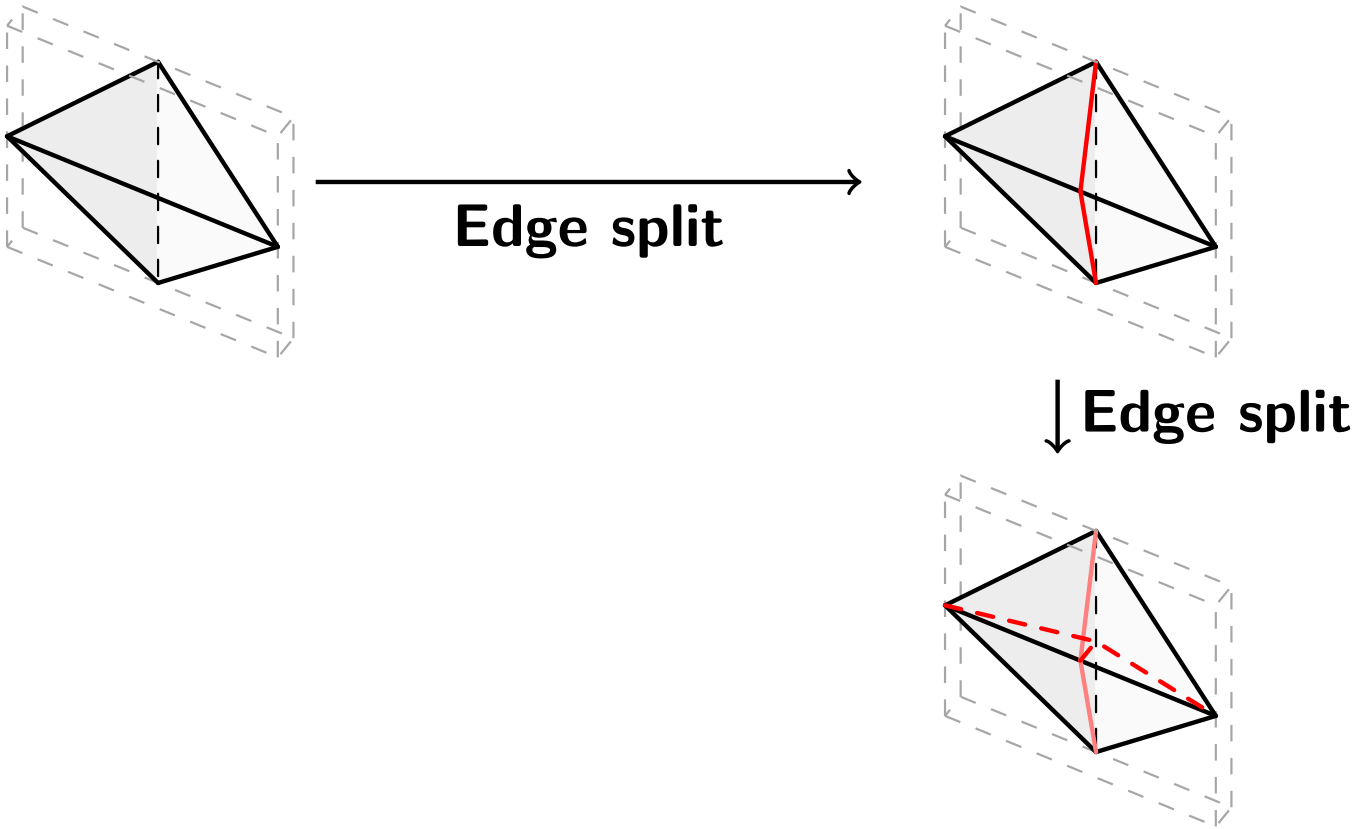


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation ⇒ Create a new operation

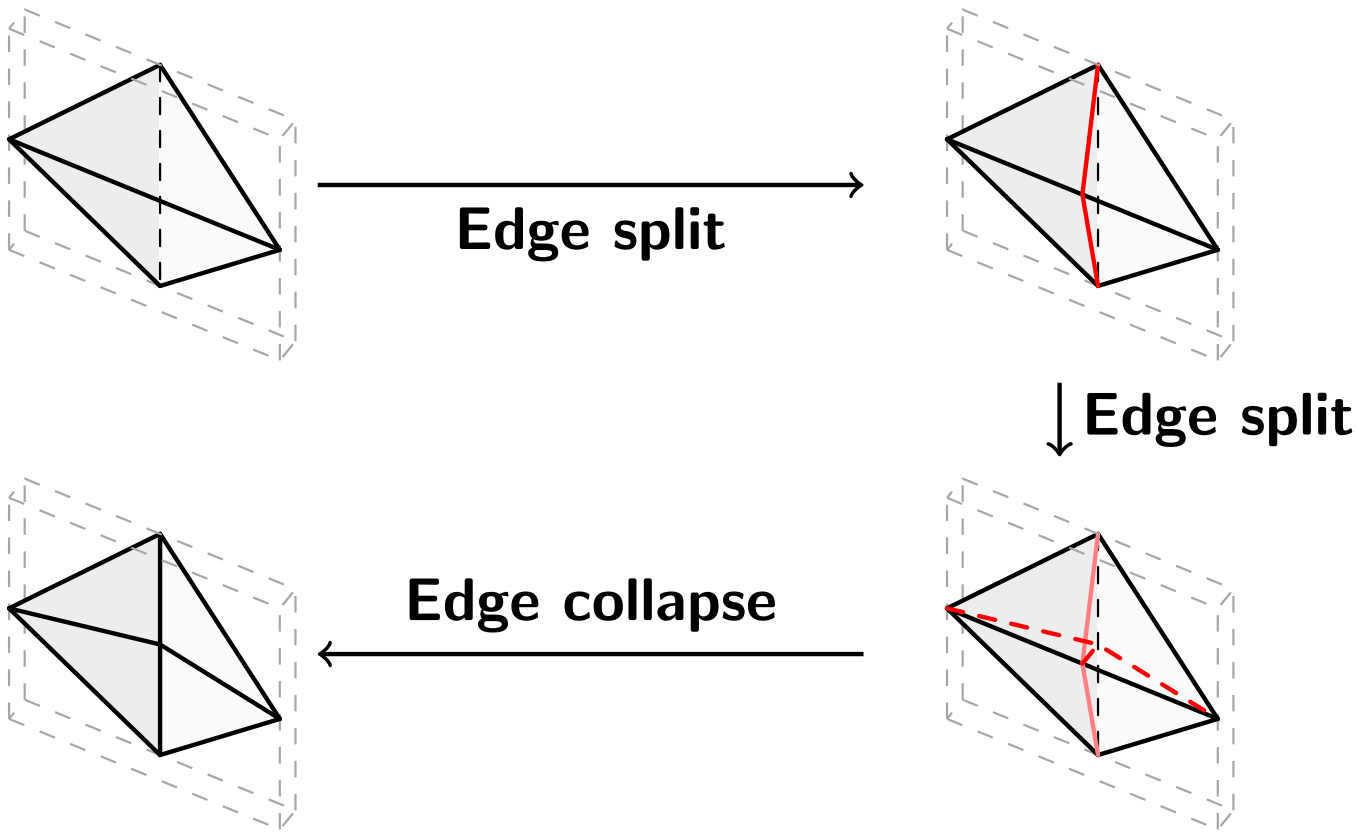


Figure: Tetrahedron collapsing process

An example

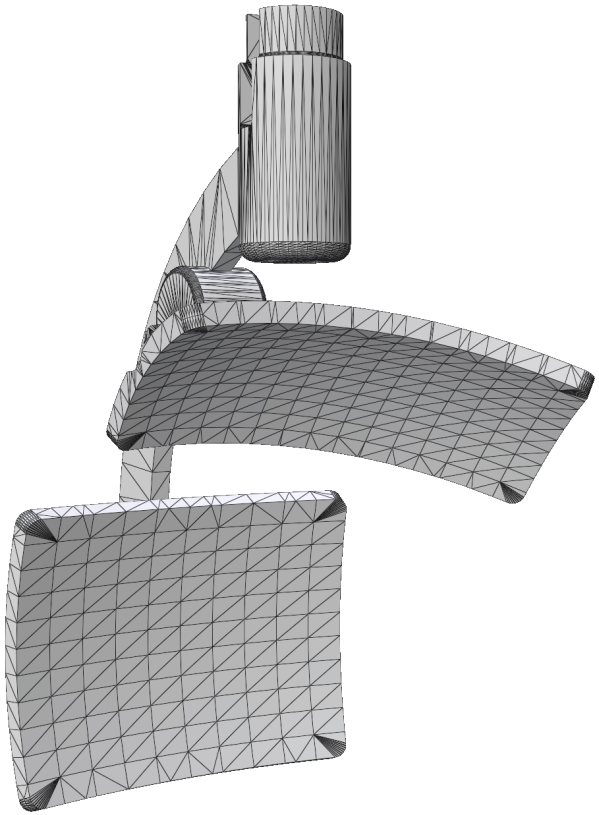
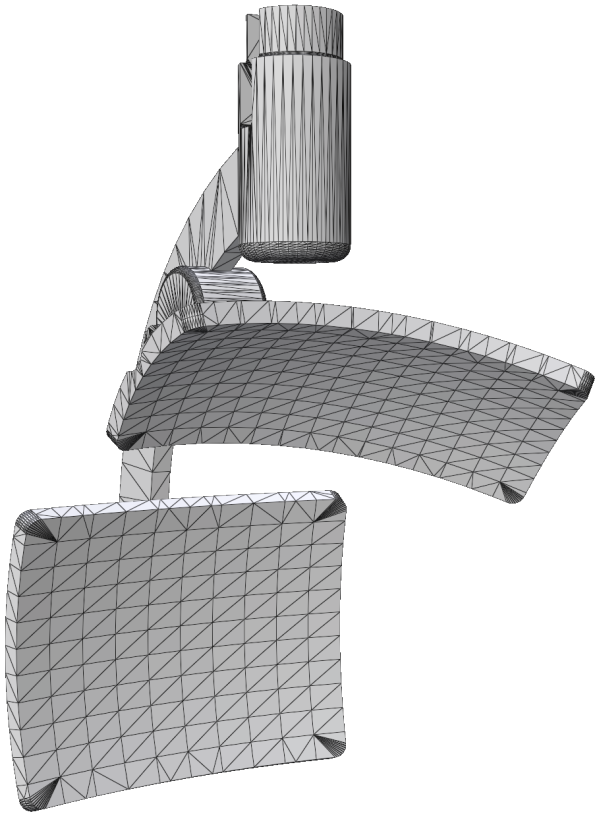


Figure: The model 112544

An example



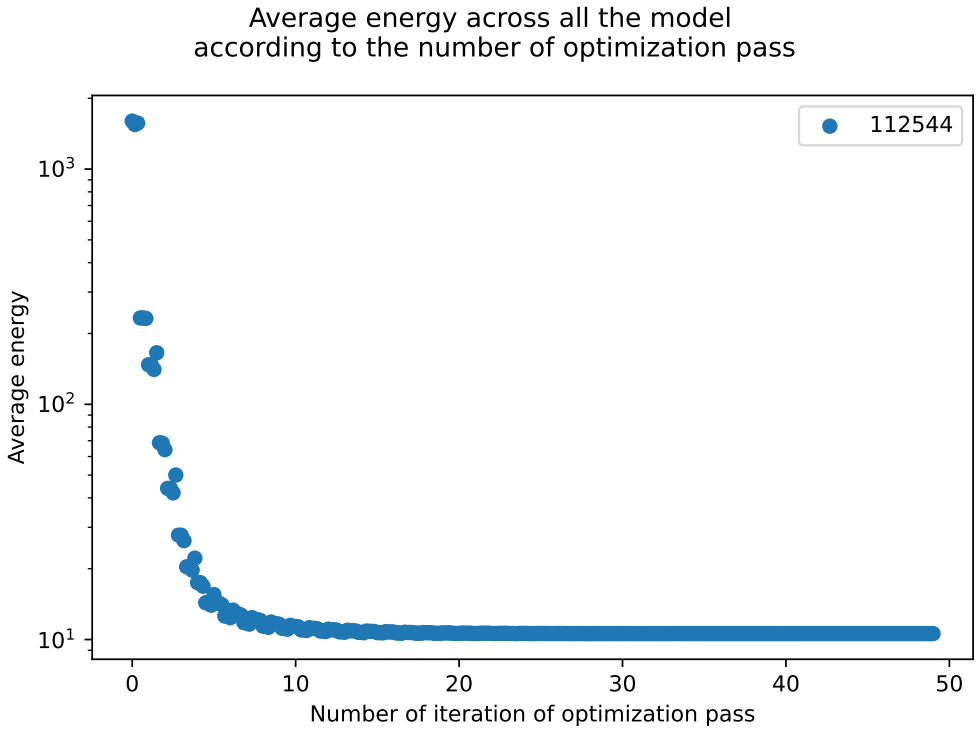
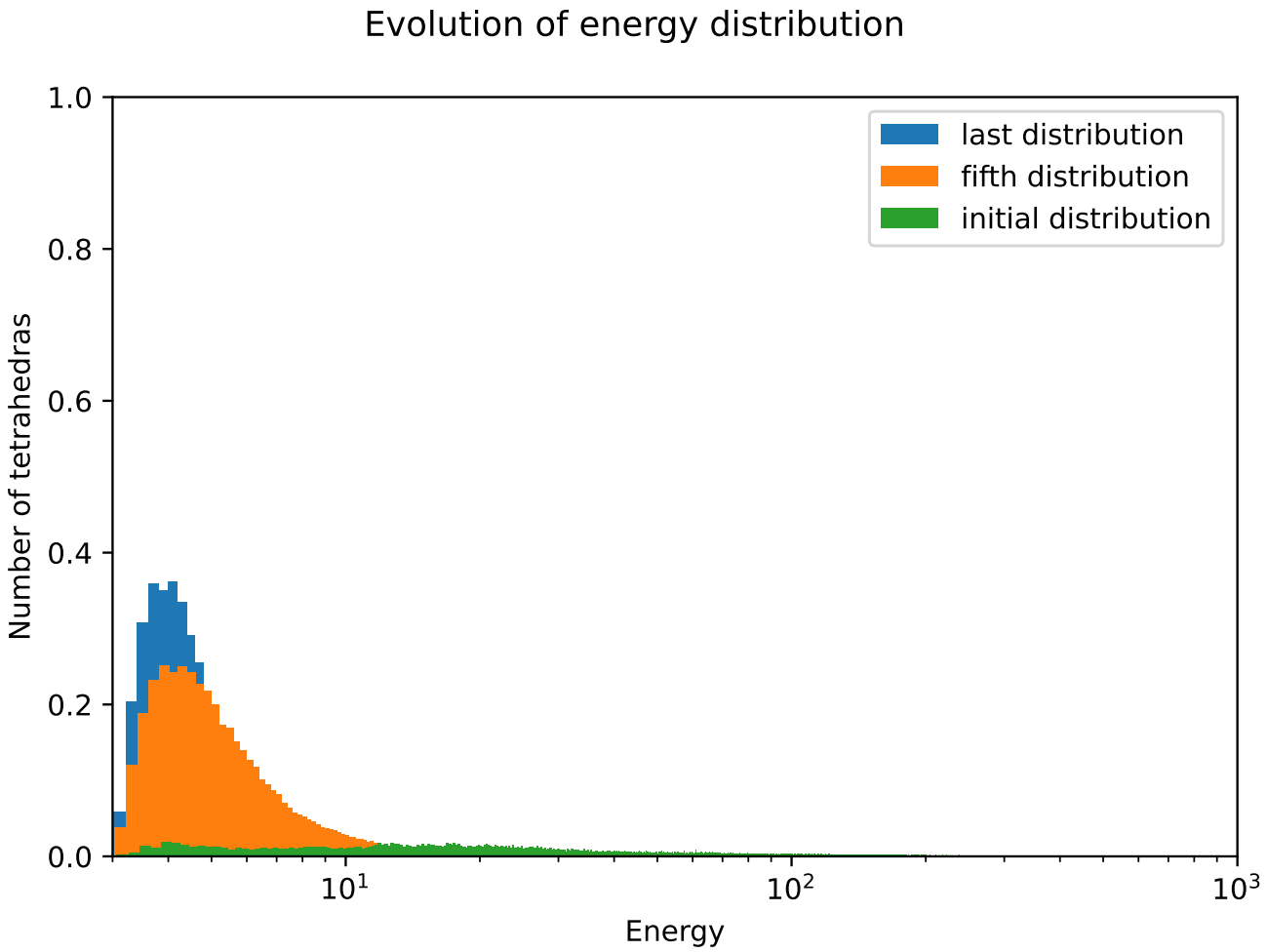


Figure: The model 112544

Evolution of the distribution



Comparison

- \+ More robust

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

Comparison

- \+ More robust
- \- Slower

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%

Table: Stats on the remaining models TetGen did not failed on

Comparison

- \+ More robust
- \- Slower
- \- Slightly less efficient

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%
Best on mean	5,35%	94,65%
Best on max	20,59%	79,41%

Table: Stats on the remaining models TetGen did not failed on

Comparison

- \+ More robust
- \- Slower
- \- Slightly less efficient

Can be explained by:

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%
Best on mean	5,35%	94,65%
Best on max	20,59%	79,41%

Table: Stats on the remaining models TetGen did not failed on

Comparison

- + More robust
- - Slower
- - Slightly less efficient

Can be explained by:

- More strict in our operations

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%
Best on mean	5,35%	94,65%
Best on max	20,59%	79,41%

Table: Stats on the remaining models TetGen did not failed on

Comparison

- + More robust
- - Slower
- - Slightly less efficient

Can be explained by:

- More strict in our operations
- Lack of time

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%
Best on mean	5,35%	94,65%
Best on max	20,59%	79,41%

Table: Stats on the remaining models TetGen did not failed on

Future work and improvements

- Optimize the speed

Future work and improvements

- Optimize the speed
  - Use more specific data structures
  - Be less restrictive

Future work and improvements

- Optimize the speed
  - Use more specific data structures
  - Be less restrictive
- Create new operations

Future work and improvements

- Optimize the speed
  - Use more specific data structures
  - Be less restrictive
- Create new operations
  - To remove needles

Future work and improvements

- Optimize the speed
  - Use more specific data structures
  - Be less restrictive
- Create new operations
  - To remove needles
- Make some operations finer

Future work and improvements

- Optimize the speed
  - Use more specific data structures
  - Be less restrictive
- Create new operations
  - To remove needles
- Make some operations finer
  - Split vertex not always at midpoint

References

H. Si.
TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator.
ACM Trans. on Mathematical Software, 41 (2), Article 11, 2015.

Y. Hu, Q. Zhou, X. Gao, A. Jacobson, D. Zorin, D. Panozzo.
Tetrahedral Meshing in the Wild.
ACM Trans. Graph. 37, 4, Article 60, 2018.